

In-class exercise: advanced SDT test suite

- Goal: practice TDD workflow on an advanced `SignalDetection` object.

- Goal: practice TDD workflow on an advanced `SignalDetection` object.
- Focus on writing tests that are correct, clear, and runnable.

In-class exercise: advanced SDT test suite

- Goal: practice TDD workflow on an advanced `SignalDetection` object.
- Focus on writing tests that are correct, clear, and runnable.
- Use this session to draft a strong baseline for homework.

- Use Python and `unittest`.

- Use Python and `unittest`.
- Put tests in:

- Use Python and `unittest`.
- Put tests in:
 - `tests/test_signal_detection.py`

- Use Python and `unittest`.
- Put tests in:
 - `tests/test_signal_detection.py`
- Run tests with:

- Use Python and `unittest`.
- Put tests in:
 - `tests/test_signal_detection.py`
- Run tests with:

- Use Python and `unittest`.
- Put tests in:
 - `tests/test_signal_detection.py`
- Run tests with:

```
python -m unittest tests/test_signal_detection.py
```

You are writing tests for an advanced `SignalDetection` object with:

You are writing tests for an advanced `SignalDetection` object with:

- `rates(hit_rate, false_alarm_rate)`

You are writing tests for an advanced `SignalDetection` object with:

- `rates(hit_rate, false_alarm_rate)`
- SDT metrics (`d_prime`, `criterion`)

You are writing tests for an advanced `SignalDetection` object with:

- `rates(hit_rate, false_alarm_rate)`
- SDT metrics (`d_prime`, `criterion`)
- operators (`__add__`, `__sub__`, `__mul__`)

You are writing tests for an advanced `SignalDetection` object with:

- `rates(hit_rate, false_alarm_rate)`
- SDT metrics (`d_prime`, `criterion`)
- operators (`__add__`, `__sub__`, `__mul__`)
- plotting methods (`plot_sdt`, `plot_roc`)

Create tests for:

Create tests for:

- `hit_rate()` in normal cases

Create tests for:

- `hit_rate()` in normal cases
- `false_alarm_rate()` in normal cases

Create tests for:

- `hit_rate()` in normal cases
- `false_alarm_rate()` in normal cases
- `d_prime()` against known values

Create tests for:

- `hit_rate()` in normal cases
- `false_alarm_rate()` in normal cases
- `d_prime()` against known values
- `criterion()` against known values

Create tests for:

- `hit_rate()` in normal cases
- `false_alarm_rate()` in normal cases
- `d_prime()` against known values
- `criterion()` against known values

Create tests for:

- `hit_rate()` in normal cases
- `false_alarm_rate()` in normal cases
- `d_prime()` against known values
- `criterion()` against known values

Use explicit expected values and tolerances where needed.

Create tests for:

Create tests for:

- invalid constructor values (e.g., negative counts, wrong types)

Create tests for:

- invalid constructor values (e.g., negative counts, wrong types)
- invalid operator arguments

Create tests for:

- invalid constructor values (e.g., negative counts, wrong types)
- invalid operator arguments
- internal consistency after operations

Create tests for:

- invalid constructor values (e.g., negative counts, wrong types)
- invalid operator arguments
- internal consistency after operations

Create tests for:

- invalid constructor values (e.g., negative counts, wrong types)
- invalid operator arguments
- internal consistency after operations

Your tests should verify that failure modes are clean and informative.

Create tests for:

Create tests for:

- `__add__` elementwise count combination

Create tests for:

- `__add__` elementwise count combination
- `__sub__` elementwise subtraction

Create tests for:

- `__add__` elementwise count combination
- `__sub__` elementwise subtraction
- `__mul__` scalar scaling

Create tests for:

- `__add__` elementwise count combination
- `__sub__` elementwise subtraction
- `__mul__` scalar scaling
- non-mutation of original objects when new objects are returned

Create tests for:

Create tests for:

- `plot_sdt()` returns matplotlib objects and labels key elements

Create tests for:

- `plot_sdt()` returns matplotlib objects and labels key elements
- `plot_roc(sdt_list)` handles a sequence of objects

Create tests for:

- `plot_sdt()` returns matplotlib objects and labels key elements
- `plot_roc(sdt_list)` handles a sequence of objects
- `plot_roc` includes `(0,0)` and `(1,1)`

Create tests for:

- `plot_sdt()` returns matplotlib objects and labels key elements
- `plot_roc(sdt_list)` handles a sequence of objects
- `plot_roc` includes `(0,0)` and `(1,1)`
- plotting can be tested without interactive display

- Use descriptive test names.

- Use descriptive test names.
- Keep one behavior per test.

- Use descriptive test names.
- Keep one behavior per test.
- Use direct, readable assertions.

- Use descriptive test names.
- Keep one behavior per test.
- Use direct, readable assertions.
- Include at least one edge case per feature group.

Before the end of class, submit a draft test suite that includes:

Before the end of class, submit a draft test suite that includes:

- a runnable `tests/test_signal_detection.py`

Before the end of class, submit a draft test suite that includes:

- a runnable `tests/test_signal_detection.py`
- at least one passing test in each of the four parts

Before the end of class, submit a draft test suite that includes:

- a runnable `tests/test_signal_detection.py`
- at least one passing test in each of the four parts
- at least one deliberate failure you fixed during class

- Which tests gave you the highest confidence?

- Which tests gave you the highest confidence?
- Which behavior was hardest to test and why?

- Which tests gave you the highest confidence?
- Which behavior was hardest to test and why?
- What assumption about the SDT object did your tests expose?